

Evaluating a Hybrid Static, Evolutionary ML, and LLM-Based Pipeline for Code Evolution Analysis

Animesh Giri

Dept. of CSE
PES University)

Bengaluru, India

animeshgiri@pes.edu

Shri Karthik A

Dept. of CSE
PES University)

Bengaluru, India

shrkrthk200518@gmail.com

Ankana Mandal

Dept. of CSE
PES University)

Bengaluru, India

ankana10.mandal@gmail.com

Anushka Mandal

Dept. of CSE
PES University)

Bengaluru, India

anushka10.mandal@gmail.com

Abstract—Software files undergo developments that are complex and are not easily explainable using snapshot-based measures. This paper presents a hybrid pipeline that evaluates the time evolution of each code file with solid and understandable evidence that can help us draw a clear picture of how it evolved. The system extracts commit-level static metrics from Python and JavaScript/TypeScript repositories, constructs temporal feature representation which measures churn, complexity, maintainability and volatility. Then, it performs unsupervised clustering and anomaly detection to detect any unusual evolution patterns

In order to explain these trends, the pipeline uses a constrained large language model, which is applied as an interpretative layer. The explanations are descriptive in nature, based on the retrieved code diffs, and precomputed signals and controlled by explicit rules that forbid speculation, prediction, or prescriptive advice. The statistical detection/semantic interpretation separation approach provides traceable and understandable explanations of file-level evolution. The qualitative analysis of open-source repositories shows that the approach emphasizes the non-obvious structural patterns but is verifiable and reproducible.

Index Terms—static code analysis, temporal code metrics, anomaly detection, neuro-symbolic systems, clustering, code diffs, explainable AI, large language models.

I. INTRODUCTION

A. Problem Context

Software systems evolve continuously with commits that modify their structure, behavior and dependencies at file level. Even though this dynamic nature is inherent, most code quality and maintainability tools rely on a snapshot-based static analysis, which gives metrics such as complexity or maintainability at any given phase in time. Although such snapshots are useful for local assessment, they obscure file evolution by masking trends like progressive simplification, increased complexity, or patterns of change and fluctuation.

Large language models (LLMs) have been used to analyze software tasks since they can provide natural-language descriptions. Nevertheless, the methods of LLM do not usually have an explicit basis in version-control history or quantifiable measurements, which casts doubt on unprovable statements, conjectural theory, and hallucinated explanations. Consequently, both the conventional static tools and the LLM-only models do not offer a credible explanation of the mechanisms and reasons for software artifact evolution.

B. Core Limitations of Existing Approaches

The current methods exhibit complementary weaknesses. Snapshot-based static analysis tool of evolution summarize the rich evolutionary histories into shallow, pointwise indicators, which are noisy and hard to interpret longitudinally. Machine-learning-based systems are able to identify statistical patterns or deviations in software measures that are often semantically opaque, and it is not clear why particular files receive this label. On the other hand, LLM-only systems produce fluent texts, but are not constrained by any formalities or grounded in evidence, which makes their outputs hard to justify and unsuitable to use analytically. Such constraints indicate that none of the single paradigm static analysis, machine learning or language models can be adequate to describe software evolution in a reliable and understandable way.

Key Insight This work is motivated by the observation that software files are dynamically evolving objects rather than static artifacts. The evolution of software has to be reasoned through time patterns of change and have to draw explicit line between statistical experimentation and semantic meaning. Separating what is perceived to change and how these changes are described allows combining both quantitative rigor and interpretability without engaging hypothetical reasoning.

C. Contributions

The main contributions of this paper are:

- A method for modeling **file-level changes over time** using time-based representations of static code metrics taken from version-control history.
- A **interpretable unsupervised learning approach** that groups files and detects unusual change patterns without using labeled data.
- A **restricted large language model explanation layer** that is used only to explain detected patterns and follows strict grounding rules.
- A **structured qualitative comparison of four analysis approaches**, showing how time-based modeling, unsupervised detection, and constrained explanations improve the clarity and reliability of software evolution analysis.

II. RELATED WORK

Rasheed et al. [1] and Fan et al. [2] propose LLM-powered systems for automated code review systems for bug detection, code smells and other optimizations based on static snapshots of code through GPT-based agents. Rasheed et al. utilize multi-agent natural-language feedback for fostered best practices and developer learning where Fan et al. employ a ReAct-style reasoning and tooling approach to higher accuracy bug detection with lower false negatives. Thus, while both are strong approaches for snapshot-level assessments, they also rely on a single code version, added LLM execution overhead, and do not consider long-term repository growth or temporal inconsistencies.

Arthur-Jozsef Molnar and Simona Motogna [3] study long-term assessment of maintainability across three open-source Java programs over 18 years based on Maintainability Index (MI), Technical Debt Ratio (TDR) and the ARiSA Compendium, all backed by manual inspection. Their results observe behaviors over time from early instability to late stabilization of mature systems and in more direct causation, linking refactorings to maintainability over long periods of time. Yet their approach relies on rule-based metrics and manual determination as opposed to clustering of like growth patterns and natural-language summary.

Venkata Sai Sravya Sambaturu [4] adopts a longitudinal case study of the TestNG Java testing framework over multiple versions, extracting object-oriented metrics applied to longitudinal growth patterns using statistical summary and visual determination. The results show system stabilization and maintained focus post refactoring and architectural changes. Yet this approach relies on a single system and finds descriptive statistics without clustering, anomaly detection or growth patterns beyond interpretive summary of like evolution.

Mahsa Salehi and Lida Rashidi [5] look at unsupervised anomaly detection methods for evolving data (data streams, dynamic graphs) with a focus on the capacity to assess non-stationary, non-labeled behaviors. While relevant to unsupervised analysis in a temporal context, the survey is general data mining oriented, not specifically software repository or code-based, evolving metrics.

Ozturk et al. [6] and Pandini et al. [7] explore LLMs as substitutes for or complements to traditional static analysis, applying LLMs to vulnerability assessment, lack of refactoring detections, and architectural smell assessments. Ozturk et al. compare ChatGPT with various static analyzers on vulnerability detection via OWASP guidelines in PHP applications, finding higher recall but drastic increases in false positives, assessments from a snapshot-based approach. Pandini et al. find LLM-refactoring recommendations for architectural smells through retrieval-augmented contexts where higher contextual loading improves prescriptive suggestions. While differing in tasks, both seek static snapshots with all prescriptive outputs (fixes/refactorings) difficult to ascertain through past code development.

Overall, prior research contributes to LLM-based static

analysis, code review, refactoring results and vulnerability detection while foundational work has dug into metrics over time and maintainability growth patterns over time. Yet few such approaches rely on Git history extraction, unsupervised pattern identification and anomaly detection with constrained LLM explanation for purely descriptive software evolution. This gap is filled by focusing on longitudinal evolution of repositories with defensible, evidence-based syntheses of differing maintenance patterns that avoids judgments, predictions and free-text generation.

III. SYSTEM OVERVIEW

Figure 1 presents the general structure of the proposed end-to-end pipeline. the pipeline begins with a process of repository data extraction, which assembles commit metadata, file change history, and code diffs needed to compute static metrics on each instance of a file. this metrics are a chronological aggregation of these metrics on per-file bases to build temporal representations that describe the changes in code properties across time.

These temporal signals are analyzed using an unsupervised machine learning layer which detects groups of files with similar evolution patterns and identifies files whose evolution trajectories not aligned with repository-wide patterns. The resulting file and cluster level summaries are encoded as structured analytical signals and sent to a context construction part. This element retrieves all the pertinent historical code diffs and snippets in the repository through a retrieval-augmented generation (RAG) layer.

Finally, the large language model (LLM) layer takes this curated context and precalculated analytical signals only, consume them to produce descriptive explanations of observed patterns of evolution. The LLM neither analyzes, scores, nor makes decisions, and it has no effect on upstream results.

The pipeline is guided by the following design principles. First, longitudinal commit histories are analyzed to get trends, stability and volatility not visible in single version snapshots. Second, unsupervised learning is utilized to prevent the use of labeled data or predictive goals and allows the characterization of the evolution behavior as opposed to deriving the defects or future results. Third, the LLM is specifically limited to explanation; any numerical signals, labels, and comparisons are generated deterministically beforehand. Such a split enhances readability, repeatability and tracking of the resulting explanations.

The system results in file-level summaries of the single evolution behaviour and repository-level syntheses which compare the maintenance strategies of components. All results are descriptive in character and are aimed at describing the way and locations of concentration of change over time, not giving prescriptions, or rating of code quality, or anticipating defects. Each of these phases of the pipeline is outlined subsequently.

IV. STATIC ANALYSIS LAYER

The static analysis layer has the task of deriving reproducible file-level code metrics from the repository. It analyzes

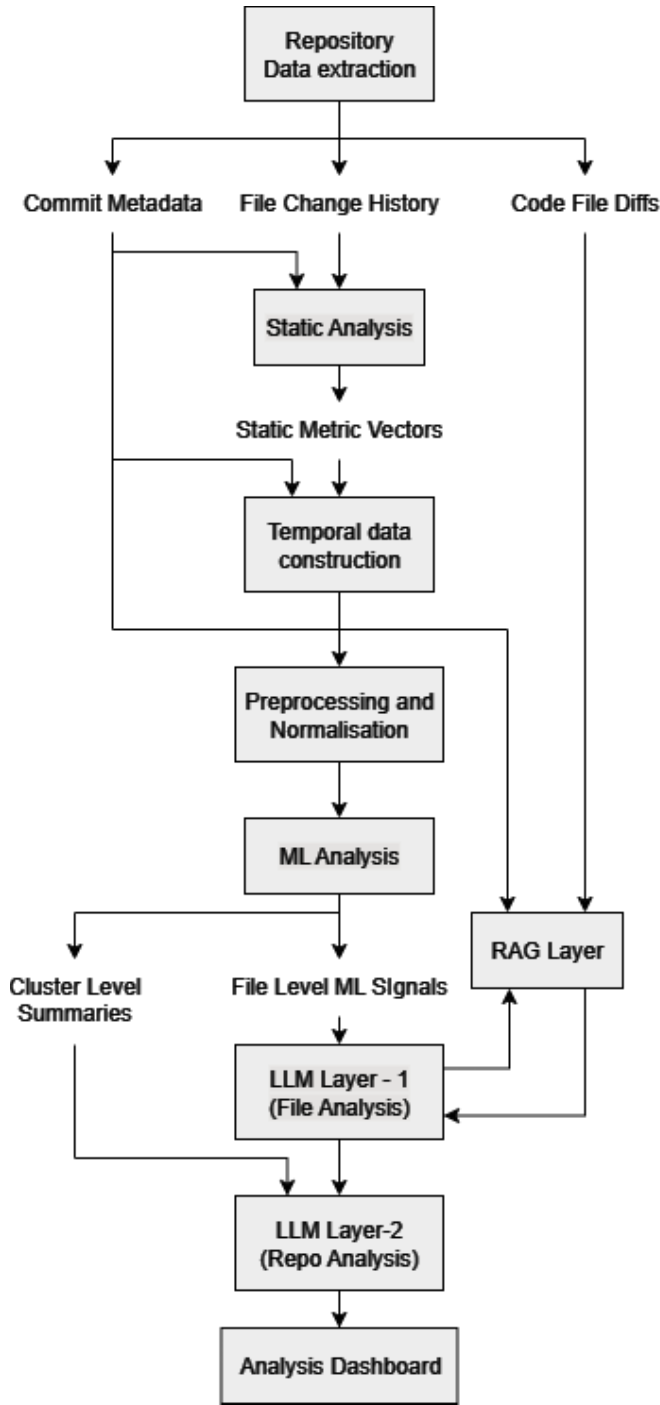


Fig. 1. End-to-end system architecture for temporal software evolution analysis. Static metrics are extracted per commit, aggregated into temporal file representations, analyzed using unsupervised learning, and translated into descriptive explanations through a constrained LLM layer grounded via retrieval-augmented context.

the commit history, and the metrics are calculated at the file and commit granularity. The layer does not strive to understand either code quality or behavior but generates structured signals, which become the input to later temporal and machine learning analysis.

Static metrics are calculated on all the files that were changed in the same commit, for each commit in the repository. Measurements are also calculated on the respective versions of such files in the immediate parent commit, making the revisions within a series of revisions easy to compare. This before-after mapping prepares the system to record not only the absolute metric values but the size and direction of the change brought about by each commit.

The measures that are extracted are standard structural metrics such as the number of lines of code, cyclomatic complexity, maintainability index (where available), number of functions, and number of imports. Each modified file is logged by the system with the values of the metrics before and after modification, their deltas, and the commit metadata. This representation maintains finer-grained change-information as well as snapshot-level structure without aggregating or interpreting metrics at this phase.

Although static metrics are calculated on a commit-by-commit basis, they are not examined independently. Rather, the static analysis layer analysis generates longitudinal record of metric variation of each file that will subsequently be converted into temporal representations and an elevated degree of behavioral features. In that sense, static analysis is used purely as an extraction phase of evidence, which offers invariably language independent hints, but leaves the interpretation of behavior to subsequent parts of the pipeline.

V. EVOLUTIONARY ML ANALYSIS

This is the machine learning component that will define file-level evolution over time. It is important to highlight interpretability, determinism, and reproducibility; thus, no predictive modeling or black-boxing occurs. All that is analyzed comes from intentionally created temporal features from static code metrics and version-control changes.

A. Temporal Modeling of File Metrics

Each source file can be conceptualized as a time series of metrics taken from static analysis on a per-commit basis. After linking static metrics to the time of the commit, all metrics are summed up per repository and file path and temporally ordered. Therefore, one may assume that a file is a dynamic entity, not static, and dynamic over time is the condition.

In order to confirm a strong enough temporal signal, any file with a number of commits below a set threshold is removed from further analysis. This is to prevent false positive artifacts from files with too weak of a historical pattern and to ensure that all time series under consideration operate as ideal cases of actual dynamic change over time.

Such a temporal framing allows for subsequent analyses to capture change over time, stability, and instability—not information that would be conveyed through a snapshot assessment.

B. Feature Engineering

We take each file's time series and make a feature vector with a fixed length. This sums up how things change over

time. We keep the features easy to understand and linked to the original data. They are:

- **Change intensity features**, such as churn rate, capturing the frequency and magnitude of modifications.
- **Central tendency features**, including mean values of complexity and maintainability measures.
- **Trend features**, computed as directional changes over time, indicating increasing, decreasing, or stable structural behavior.
- **Volatility features**, measuring variability across commits to capture irregular or unstable evolution.
- **Structural size indicators**, such as function and import counts, providing contextual scale.

We calculate all features using clear, fixed statistical operations. Before we do clustering or anomaly odd cases, we scale the features so they are fair to compare, even if files are very different.

C. Clustering of Evolution Patterns

Unsupervised clustering takes place to join together files with the most similar evolutionary trends. This is not for classification purposes, but instead an exploratory feature that allows the most prominent trends of repository evolution to spring from the data. K-means clustering occurs because it's simple to understand, means what it says, and is replicable. A silhouette analysis across a sufficiently small and reasonable range of values exists to ascertain the number of clusters. Each cluster contains a significant, recurring type of evolution, such as low-change files with consistent complexity throughout certain regions or high-change files that show gradual reduction. Cluster assignment is also completely interpretable based on its value characteristics, allowing the researcher to see exactly why files are similar enough to be in a cluster.

D. Anomaly Detection via Isolation Forest

Anomaly detection is executed through Isolation Forest to find files that are unlikely to evolve like their counterparts over time. Anomaly detection exposes observations on the edge using feature distributions via a non-parametric, unsupervised setting. The fewer the number of split decisions per observation, the higher the anomaly score.

This is not an error-determination classification, but rather a file selection hierarchy. The higher the anomaly score, the more files are deemed anomalous evolution participants—over time, they have evolved in a way differently than their counterparts in the population. They are then selected for further semantic interpretation in the interpretation layer.

Post-ML Rule-Based Interpretation To maintain semantic consistency and avoid unwanted label creation post-machine learning, these interpretations are rule-based and connect the superficial characteristics of quantitative measures—whether the trend is decreasing or increasing, the churn level, the stability, the volatility—with rough descriptive labels (stable complexity, increasing complexity, high churn). These labels add no extra information but are used as regulated abstractions over the manifestation of measures. This means that

explanations given downstream remain sufficiently related to something empirically observable and that subsequent runs on the same files would receive the same interpretation. This step explicitly constrains the interpretation space, reinforcing determinism and reducing ambiguity in subsequent analysis.

VI. LLM-BASED EXPLANATION LAYER

The LLM layer is used only to turn precomputed signals into clear, human-readable explanations. It does not detect issues, assign scores, rank files, or make predictions, and it does not affect any earlier analysis. All decisions about anomalies and file groupings are made before this stage. The LLM is used only to explain results in a clear way and to avoid guessing or unclear reasoning.

A. Role of the LLM in the Pipeline

Within the pipeline, the LLM acts as a descriptive interpretation layer that works with fixed and structured inputs. It does not access raw repositories, compute metrics, assign anomaly scores, or predict future behavior. Instead, it uses only the signals produced earlier and converts them into Natural-language summaries that help explain and understand code evolution

By keeping the LLM separate from analysis, the system ensures that all conclusions are based on clear and repeatable signals. This prevents the LLM from adding hidden assumptions or unsupported judgments to the analysis.

B. Context Construction and Grounding

To ground the explanation in repository evidence, LLM is provided with a limited and carefully selected set of information from the repository to produce explanations. For each file, we retrieve relevant past code changes and code snippets and combine them with simple signals produced by the earlier machine learning step.

The LLM does not receive the full repository, raw metric values, or complete commit history. Instead, it is given only the parts directly linked to the signals being explained. This ensures that every explanation can be traced back to concrete evidence from the repository history.

C. Constraints and Hallucination Control

The explanation process is kept strict to reduce errors and maintain consistency. The LLM follows a fixed output format and is not allowed to give advice, make guesses, or express opinions. It only describes what is shown by the provided signals and retrieved code.

Each explanation is checked before being accepted. If it breaks the expected format or includes unsupported claims, it is rejected and generated again. This ensures that the LLM acts only as a controlled translation step and does not introduce new reasoning.

TABLE I
EVALUATED OPEN-SOURCE REPOSITORIES

Repository	Language	Branch	Commits	Files	Role
Flask	Python	main	287	19	In-depth
Next.js	JS / TS	canary	500	117	In-depth
Requests	Python	main	364	11	Supporting
FastAPI	Python	main	500	58	Supporting
Django REST	Python	main	491	31	Supporting
Pydantic	Python	main	500	46	Supporting
Express	JavaScript	main	488	31	Supporting
Axios	JavaScript	main	488	21	Supporting
Vue.js	JavaScript	main	491	50	Supporting

D. Output Structure and Hierarchical Synthesis

The LLM generates explanations at two levels. At the file level, it explains how each file changes over time, pointing out patterns such as steady growth, stabilization, frequent changes, or behavior that differs from the rest of the repository. At the repository level, it combines file-level explanations to compare different maintenance styles and change patterns across components.

All outputs follow a structured format and are purely descriptive. The LLM does not suggest actions, rank files, or judge code quality. Instead, it provides clear and traceable explanations that help maintainers see where changes happen, how files evolve differently, and which parts of the system show unusual behavior over time.

VII. COMPARATIVE EVALUATION

A. Experimental Setup

The analysis is performed on nine open-source repositories of Python and JavaScript/TypeScript codebases. These repositories are of varying sizes, complexity of architecture, and levels of development activity, and they offer a wide range of software evolution profiles. There was no explicit filtering to rule out test files or generated files or documentation artifacts and analysis was done across the main development branch of each repository.

For detailed qualitative comparison, two repositories are examined in depth: *Flask*, a Python web framework, and *Next.js*, a JavaScript/TypeScript framework. The Flask analysis covers commits from August 29, 2021 to March 11, 2023 on the `main` branch, comprising 287 commits and 19 files modified during the analyzed period. The Next.js analysis covers commits from October 15, 2025 to November 20, 2025 on the `canary` branch, comprising 500 commits and 117 files modified during the analyzed period.

The other repositories are employed to determine the consistency and generality in the observed patterns of evolution and dial in the pipeline to work across repositories. These repositories are not shown in detail because of space limitations, but their presence helps to cross-project validate the qualitative results of the project instead of generalizing them statistically.

TABLE II
INFORMATION AVAILABILITY ACROSS ANALYSIS MODES. THE TABLE SUMMARIZES WHICH CLASSES OF INFORMATION ARE DIRECTLY PRODUCED, IMPLICITLY SUBSUMED, OR UNAVAILABLE UNDER EACH ANALYSIS CONFIGURATION.

Information Available	Static	ML + Static	LLM + Static	Hybrid
Snapshot-level metrics	✓	✓	✓	✓
Temporal metric representations	×	✓	✓	✓
Snapshot-level explanations	×	×	✓	○
Temporal evolution explanations	×	×	×	✓

0.3em **Legend:** ✓ directly produced; ○ implicitly subsumed within temporal explanation; × not available.

B. Compared Analysis Modes

To assess the value of each of the layers proposed in the pipeline, four analysis modes are compared, which include a gradual addition of the static analysis, the temporal modeling, and the explanation. Instead of comparing the tools with external benchmarks, this comparison seals off the role of each layer in the same pipeline, allowing a controlled evaluation of how each analytical power influences the characterization of software evolution that results.

Static-only analysis works on the snapshot level of static metrics computed on a per-file basis at a single commit. This mode gives a structural measurement that does not pool information across revisions and is not a temporal behavior model that generates explanatory summaries.

ML-only analysis takes into account the temporal aggregation of file-level metrics of commit histories and uses unsupervised learning to define regular patterns of evolution and deviations. This mode generates structured results including cluster assignments, anomaly scores and temporal metric trends like churn rate, complexity slope but not natural-language explanations.

LLM-only analysis applies a large language model directly to the current code snapshot and associated diffs without access to temporal features, clustering results, or anomaly signals and this mode produces natural-language explanations based on local context alone without access to ML layer outputs as described before.

Hybrid analysis combines all layers of the pipeline. The static metrics are combined and transformed into temporal representations, analyzed using unsupervised learning, and is later translated into descriptive explanations by a constrained LLM layer grounded in retrieved historical context. This mode in turn showcases the full proposed approach and acts as the primary subject of evaluation.

This layer-wise comparison enables comparison of how temporal modeling and constrained explanation in the LLM can alter the structure, specificity, and grounding of analysis outputs, without relying on predictive accuracy or external benchmarks.

TABLE III

DISTRIBUTION AND DESCRIPTIVE PROFILES OF EVOLUTION CLUSTERS IDENTIFIED IN THE FLASK REPOSITORY. CLUSTER DESCRIPTIONS ARE DERIVED POST HOC FROM DOMINANT TEMPORAL METRIC TRENDS AND ARE INTENDED SOLELY AS DESCRIPTIVE CHARACTERIZATIONS.

Cluster ID	Files	Share	Dominant Temporal Characteristics
C0	18	82%	High churn, stable complexity
C1	1	5%	Low churn, growing complexity, high volatility
C2	1	5%	High churn, stable complexity
C3	2	9%	Moderate churn, declining complexity

C. Qualitative and Structural Comparison

We evaluate the four analysis modes through qualitative and structural comparison rather than quantitative performance metrics, focusing on how each mode characterizes repository evolution and what information it reveals or obscures. Two repositories are examined in depth: Flask, representing a relatively stable and mature codebase, and Next.js, representing a large, high-churn, multi-subsystem project.

1) *Flask*: Flask is a relatively stable repository with a reasonable level of churn and well-defined maintenance tasks, providing a sensible target for evaluating the impact of analysis mode on routine evolution in the absence of volatile architecture.

Under the *static-only* mode, analysis yields per-commit, file-level static metrics that identify localized changes in the static-only snapshots. In the Flask repository (306 commits, 19 files), these metrics identify change locations, but are fragmented and difficult to interpret in bulk. As each metric is meaningful only in the context of its own isolated commit snapshot, static-only analysis cannot be used to characterize file behavior over time or disentangle routine evolution from routine maintenance.

The *ML-only* mode aggregates file-level metrics over the commit history so that files with different temporal behaviors can be separated. As seen in Table III, the majority of the files (82%) are part of the cluster that displays high churn and constant complexity, meaning they are consistently and regularly maintained. The other, smaller clusters indicate the presence of less typical file behaviors that exhibit a structure such as decreasing complexity or more volatile files. As seen in Table IV, these clusters also differ in their dominant churn and complexity values, meaning it is still possible to find rare or non-typical ways in which files have developed over time, regardless of the lack of attributed meaning. This implementation of time makes the regenerated dependency and documentation files less significant and highlights the true application files whose evolutionary pattern departs from the typical behavior of files within this repository.

On the other hand, the *LLM-only* mode does give correct explanations for individual commit-level activities but local and does not provide a higher-level abstraction for repository-level activities. The explanations generally correctly identify

TABLE IV

DOMINANT INTERNAL TEMPORAL TRENDS WITHIN EVOLUTION CLUSTERS FOR THE FLASK REPOSITORY, BASED ON FILE-LEVEL CHURN AND COMPLEXITY TREND DISTRIBUTIONS.

Cluster ID	Dominant Churn Level	Dominant Complexity Trend
C0	High (10/18 files)	Mixed, leaning stable
C1	Low (1/1 file)	Increasing (1/1 file)
C2	High (1/1 file)	Stable (1/1 file)
C3	Moderate-High (2/2 files)	Decreasing (2/2 files)

the import replacement and dependency update activities; however, document changes, generated req files, and code refactor activities are all treated equally. So, when attempting to elevate to a higher level of abstraction (repository-level), it simply leaves the activities at a level of abstraction and labels the repository activity as "maintenance without any structural weighting of the different types of activities.

The *hybrid* model builds upon temporal data from the ML layer and evidence-based explanations to teach the LLM how to behave. By leveraging the ML layer to focus the explanation on structurally significant files, the LLM can be deployed selectively (as opposed to universally) in the repository. This selective deployment of the LLM will both improve the quality of the explanation's understanding by filtering low-signal explanations, as well as limit the invocation of the LLM for low-signal files, thus limiting the invocation of the LLM per repository and making each API call more resource-efficient for utilizing resources accurately, relevantly, and effectively. In addition, explanations generated by this model do not speculate as to how actions should be taken in the future, but reflect how actions were taken in the past and how those actions relate to the behavior exhibited over the long-term within a file.

2) *Next.js*: Next.js is a big, high-churn repo with lots of subsystems evolving in parallel, so a great playground for snapshot and temporal approaches.

In *static-only* mode, analysis shows many modified files in all sorts of apparently-unrelated places in the repo, from compiler internals in the turbopack compiler to tooling in the user-facing parts. Since metrics are computed per commit and per-file snapshot, this is to be expected, but does not filter out normal maintenance activity from what might be non-trivial ongoing structural refactorings, resulting in a rather fragmented view of what's going on in the repo.

The *ML-only* mode consolidates file-level metrics across the commit history, allowing evolution signatures to vary by subsystem. As shown in Table V, the majority, 86% of files, belong to a large cluster with typical churn and complexity remaining constant, while the small clusters contain files with churn and complexity decreasing. As also demonstrated in Table VI, the fact that such files with decreasing complexity and increased churn are prevalent in the smaller clusters, relative to those with typical churn and constant complexity, as shown in Table V, suggests localized refactoring missed by

TABLE V

DISTRIBUTION AND DESCRIPTIVE PROFILES OF EVOLUTION CLUSTERS IDENTIFIED IN THE NEXT.JS REPOSITORY. CLUSTER DESCRIPTIONS ARE DERIVED POST HOC FROM DOMINANT TEMPORAL METRIC TRENDS AND ARE INTENDED SOLELY AS DESCRIPTIVE CHARACTERIZATIONS.

Cluster ID	Files	Share	Dominant Temporal Characteristics
C0	13	11%	High churn, declining complexity
C1	101	86%	Moderate churn, stable complexity
C2	3	3%	Moderate churn, declining complexity, high volatility

TABLE VI

DOMINANT INTERNAL TEMPORAL TRENDS WITHIN EVOLUTION CLUSTERS FOR THE NEXT.JS REPOSITORY, BASED ON FILE-LEVEL CHURN AND COMPLEXITY TREND DISTRIBUTIONS.

Cluster ID	Dominant Churn Level	Dominant Trend	Complexity
C0	High (9/13 files)	Stable to decreasing	
C1	Low–Moderate (69/101 files)	Stable (88/101 files)	
C2	Moderate–High (3/3 files)	Decreasing (3/3 files)	

snapshot analysis. The temporal pattern of the divergence of the core compiler from the peripheral tooling does not require semantic analysis for its identification.

The *LLM-only* mode produces rich, spatially consistent explanations for individual commits, correctly outlining refactorings, additions of new components, and tests. Because, however, file selection is based on snapshot rather than temporal heuristics, explanations are interleaved across subsystems with equal (snapshot) prominence. The repository-level synthesis is therefore descriptive but shallow, lacking awareness of recurrence, persistence, and structural weighting.

The *hybrid* strategy combines the ML-based temporal signals with a constrained explanation. By constraining the LLM analysis to files associated with the smaller high-churn or low-complexity clusters, the explanation is focused on the structurally anomalous parts of the repo rather than attempting to explain all modified files. This results in a more structurally coherent explanation and avoids unnecessary LLM invocations while maintaining that the explanations are descriptive (not prescriptive) and code-contextualized.

3) *Summary*: Across both the repositories, static analysis finds change, ML-based temporal modeling emphasizes repo structural evolution, and LLM-only explanations ends up providing ungrounded localized semantics. Hybrid does all, leveraging all signals to constrain the LLM based explanations with ML signals to give structured repo evolution insights while making optimal use of the LLM.

VIII. THREATS TO VALIDITY

A. Repository Selection Bias

The results are derived from a set of real, open-source repositories in Python and JavaScript/TypeScript: Flask, Requests, FastAPI, Django REST Framework, Pydantic, Express, Axios, Next.js, and Vue.js. They are chosen to represent systems

of different sizes, architectures and patterns of evolution, however they do not represent all classes of software systems. Furthermore, we do not sample systems in proprietary code bases, insignificant, small projects with limited reach or time and systems of a safety-critical nature. Therefore, the evolution patterns observed may not be true across all fields of software engineering and evolution.

B. Subjectivity in Unsupervised Machine Learning

Results are derived from unsupervised learning methods for clustering and anomaly detection, meaning without labels or ground truth. Therefore, results are based on intentional selections during the process—feature selection, normalization factors, and parameter thresholds—although applied consistently across all repositories, with an emphasis on transparent results. Different feature selections or parameter ranges could yield different clusters or outliers. For this reason, results should be treated as qualitatively defined trends of observed and not necessarily categorized trends.

C. Non-Determinism of Large Language Models

The explanation element relies on a large language model to generate text for the detected patterns. Be advised, a bit of variation in generation will occur from run to run even with the strict prompting constraints, low-temp settings and cut-off for validated output, but this will not impact the reproducibility of the scored assessment, clustering findings, or deviations generated in a non-random fashion.

D. Lack of Large-Scale Human Evaluation

The evaluation is not based on a human-oriented usability test, but on systematic accuracy and claims with evidence. There was no large user study into developers’ attitudes towards, comprehension and response to, the provided rationale, so findings are methodologically valid and interpretively reasonable.

IX. CONCLUSION AND FUTURE WORK

This paper proposed a new approach for studying how software projects change over time by clearly separating the tasks of detecting changes and explaining them. The method combines static code analysis, time-based patterns identified through unsupervised machine learning, and a restricted LLM used only for explanation. This allows us to describe how code evolves without relying on predictions or judging code quality. We compare four analysis modes static-only, machine-learning-only, LLM-only, and the combined approach and show that snapshot-based analysis and unconstrained LLM explanations provide useful local details but fail to capture repository-wide structure. In contrast, our combined approach uses temporal signals and grounded explanations to describe change patterns clearly without speculation.

Several directions remain for future work. First, studies with software practitioners could help evaluate how useful the explanations are for understanding and maintaining repositories. Second, applying the approach to a larger and more

diverse set of projects would allow evaluation across different programming languages, project sizes, and development styles. Finally, although the current design already limits LLM usage by focusing on structurally relevant files, future work could explore ways to reduce computational cost while generating explanations.

REFERENCES

- [1] Z. Rasheed, M. A. Sami, M. Waseem, K.-K. Kemell, X. Wang, A. Nguyen, K. Systä, and P. Abrahamsson, “AI-powered code review with LLMs: Early results,” *arXiv preprint arXiv:2404.18496v2*, 2024.
- [2] G. Fan, X. Xie, X. Zheng, Y. Liang, and P. Di, “Static code analysis in the AI era: An in-depth exploration of the concept, function, and potential of intelligent code analysis agents,” *arXiv preprint arXiv:2310.08837v1*, 2023.
- [3] A.-J. Molnar and S. Motogna, “Longitudinal evaluation of open-source software maintainability,” *arXiv preprint arXiv:2003.00447*, 2020.
- [4] V. S. S. Sambaturu, “Evolution analysis of software quality metrics in an open-source Java project: A case study on TestNG,” *arXiv preprint arXiv:2505.22884*, 2025.
- [5] M. Salehi and L. Rashidi, “A survey on anomaly detection in evolving data [with application to forest fire risk prediction],” *ACM SIGKDD Explorations Newsletter*, vol. 20, no. 1, pp. 13–23, 2018.
- [6] O. S. Ozturk, E. Ekmekcioglu, O. Çetin, B. Arief, and J. C. Hernández-Castro, “New tricks to old codes: Can AI chatbots replace static code analysis tools?” in *Proc. European Interdisciplinary Cybersecurity Conference (EICC)*, 2023, pp. 1–10. doi: 10.1145/3590777.3590780.
- [7] G. Pandini, A. Martini, A. N. Videsjorden, and F. Arcelli Fontana, “An exploratory study on architectural smell refactoring using large language models,” in *Proc. IEEE 22nd Int. Conf. on Software Architecture Companion (ICSA-C)*, 2025, pp. 462–469. doi: 10.1109/ICSA-C65153.2025.00070.
- [8] A. Tornhill, *Your Code as a Crime Scene: Use Forensic Techniques to Arrest Defects and Architectural Decay*. Raleigh, NC, USA: Pragmatic Bookshelf, 2015.
- [9] T. Zimmermann, P. Weißgerber, S. Diehl, and A. Zeller, “Mining version histories to guide software changes,” *IEEE Trans. Software Engineering*, vol. 31, no. 6, pp. 429–445, 2005. doi: 10.1109/TSE.2005.72.
- [10] H. Gall, M. Jazayeri, and J. Krajewski, “CVS release history data for detecting logical couplings,” in *Proc. Int. Workshop on Principles of Software Evolution (IWPSE)*, 2003, pp. 13–23. doi: 10.1109/IWPSE.2003.1234007.
- [11] A. E. Hassan, “Predicting faults using the complexity of code changes,” in *Proc. 31st Int. Conf. on Software Engineering (ICSE)*, 2009, pp. 78–88. doi: 10.1109/ICSE.2009.5070502.
- [12] R. Moser, W. Pedrycz, and A. Sillitti, “A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction,” in *Proc. 30th Int. Conf. on Software Engineering (ICSE)*, 2008, pp. 181–190. doi: 10.1145/1368088.1368113.
- [13] F. Khomh, Y.-G. Guéhéneuc, and G. Antoniol, “Playing with factors: A framework for the analysis of the impact of design patterns on software quality,” *Empirical Software Engineering*, vol. 16, no. 4, pp. 469–498, 2011.
- [14] G. Bavota, A. Qureshi, S. Panichella, and G. Canfora, “An empirical investigation into the evolution of code smells,” *IEEE Trans. Software Engineering*, vol. 42, no. 10, pp. 919–944, 2016.
- [15] Y. Tian, N. Ali, A. E. Hassan, and M. Mäntylä, “On the use of large language models in software engineering: A survey of recent advances,” *ACM Computing Surveys*, 2023.